

Operating System Project

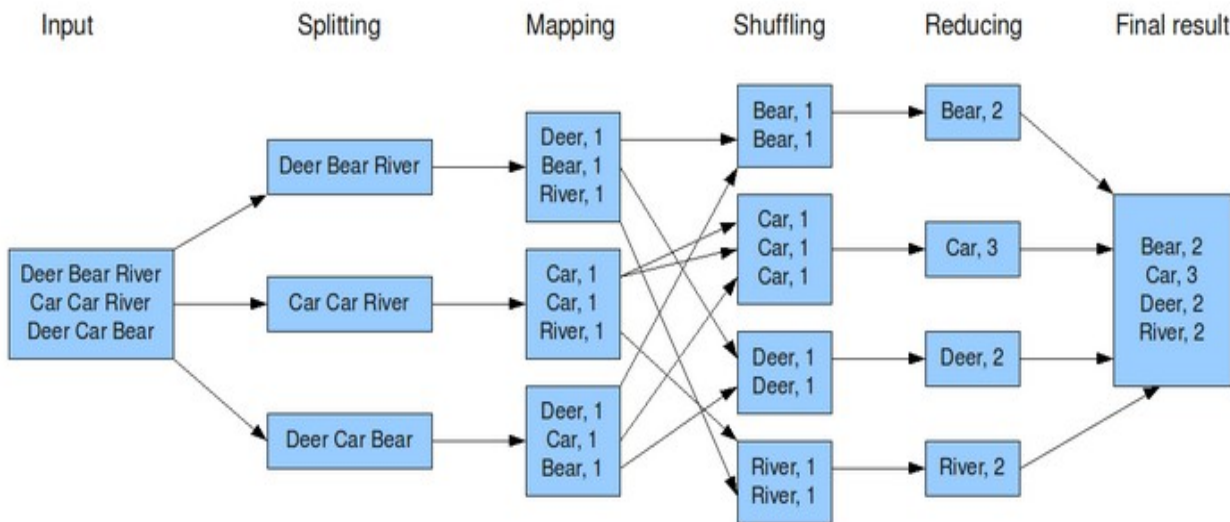
Due Date: **8th Dec 2024**, 11:59 PM (Hard Deadline)

Instructions:

- It's a group project with **3 members**.
- Make sure that you read and understand the description. If you have any questions or comments, you are encouraged to discuss your problems with your teacher.
- **Plagiarism** is strongly forbidden and will be strongly punished. If we find that you have copied from someone else or someone else has copied from you (with or without your knowledge) both of you will be punished. You will be awarded (straight zero in the project — which can eventually result in your failure) and appropriate action as recommended by the Disciplinary Committee (DC can even award a straight F in the subject) will be taken.
- Try to understand and do the project yourself. Note that you will be mainly awarded on your effort not on the basis whether you have completed the project or not.
- **No Marks** will be given if you do not use the **Operating system concepts** that you have learned during the course.
- Combine all your work in one .zip file.
- Name the .zip file as ROLL-NUM-SECTION.zip (e.g. 22i0002-B.zip).
- Submit the .zip file on Google Classroom, submission will not be accepted on any other platform for example email.
- You must follow the submission instructions, as failing to do so will get you a zero in the Project.
- Grading will be done individually.

This project involves implementing a basic MapReduce framework using fundamental operating system concepts. You will develop an understanding of how concurrent computing and synchronization work in the context of parallel processing and distributed data processing.

In this project, students will simulate a MapReduce algorithm on a single machine, to perform the **Map** and **Reduce** phases. The MapReduce breaks down a large task into smaller, independent sub-tasks that can be processed in parallel. It consists of two main operations: **Map** and **Reduce**.



1. Map Phase:

The map phase processes input data in parallel and outputs a collection of intermediate key-value pairs. The input to the map function is a set of data, which is typically split into smaller chunks that can be processed in parallel. Each chunk of data is processed by a map function, which produces key-value pairs as output.

2. Shuffle Phase:

This phase happens between map and reduce. After the Map phase, the key-value pairs are grouped by key, so that all occurrences of a particular key are together. This phase is usually handled by the system in real-world implementations.

3. Reduce Phase:

The reduce phase processes the intermediate key-value pairs and performs some aggregation or computation on them. It takes all values associated with a particular key and combines them to produce the final output. Each unique key from the map phase is sent to a reduce function, which then aggregates the associated values.

Example:**Input:**

"pizza burger pasta pasta pizza"

After Shuffle:

("pizza", [1, 1]), ("burger", [1]), ("pasta", [1,1])

Output:

("pizza", 2), ("burger", 1), ("pasta", 2)

Deliverables:

1. A complete C/C++ program that implements the MapReduce framework.
2. Word document containing flow diagram/ activity diagram of your approach and detailed explanation of the code.
3. Test cases of your code.